

MyGenWashy I2C-Protokoll fuer den ESP32

Diese Beschreibung gilt fuer beide Controller-Varianten:

- 3226TEST mit ATTiny3226
- NanoWashy mit ATmega328P / Arduino Nano

In beiden Faellen ist der Waschmaschinen-Controller der I2C-Slave und der ESP32 der I2C-Master.

1. Rollen und Busdaten

- Slave-Adresse: 0x20
- Busmodus: I2C / 7 Bit
- Empfohlene Taktfrequenz: 100 kHz
- Registermodell: registerbasiert mit internem Registerzeiger

Wichtig zum Laufzeitverhalten:

- Der Controller besitzt immer ein autonomes Basisprogramm 40 °C waschen.
- Wenn der ESP32 innerhalb der ersten 5 s nach dem Einschalten bzw. nach dem Eintritt in IDLE keine Programmdatei nach 0x10 . . 0x13 schreibt, startet automatisch das Basisprogramm.
- Reines Lesen von Statusregistern zaehlt nicht als Programmvorgabe und blockiert den Autostart nicht.
- Faellt der ESP32 waehrend eines laufenden Programms aus, laeuft das aktuelle Programm im Controller weiter.

2. Elektrische Hinweise

- Gemeinsame Masse zwischen ESP32 und Controller ist Pflicht.
- SDA und SCL muessen als Open-Drain-Bus mit Pull-ups betrieben werden.
- Bei der ATTiny-Variante ist der Bus nativ 3.3 V.
- Bei der Nano-Variante laeuft der ATmega328P zwar mit 5 V, die I2C-Pull-ups fuer den ESP32 muessen aber trotzdem auf 3.3 V liegen oder ueber einen Level-Shifter gefuehrt werden.
- Die ESP32-Pins duerfen nicht mit 5 V auf SDA oder SCL beaufschlagt werden.

Typische Verdrahtung:

- ESP32 GPIO21 -> SDA
- ESP32 GPIO22 -> SCL
- ESP32 GND -> GND

3. Registerzugriff

Schreibzugriff:

text

[register] [byte0] [byte1] ...

Lesezugriff:

- 1 Master schreibt zuerst nur die Startadresse.
- 2 Danach folgt ein Repeated-Start oder ein neuer Read-Zugriff.
- 3 Der Slave liefert ab dieser Adresse fortlaufend Registerbytes.

Beispiel: Statusblock lesen

cpp

```
Wire.beginTransmission(0x20);  
Wire.write(0x00);  
Wire.endTransmission(false);  
Wire.requestFrom(0x20, 11);
```

Beispiel: Startkommando senden

cpp

```
Wire.beginTransmission(0x20);  
Wire.write(0x10);  
Wire.write(0x01);  
Wire.endTransmission();
```

4. Register-Map

Leseregister `0x00..0x0A`

Adresse	Name	Bedeutung
---	---	---
0x00	REG_STATUS	Aktueller Zustand aus WashState
0x01	REG_ERROR	Aktueller Fehler aus ErrorCode
0x02	REG_TEMP_INT	Temperatur Ganzzahl in degC
0x03	REG_TEMP_FRAC	Temperatur Nachkommastelle in 0.1 degC
0x04	REG_RPM_HIGH	High-Byte der Motordrehzahl
0x05	REG_RPM_LOW	Low-Byte der Motordrehzahl
0x06	REG_PROGRESS	Fortschritt 0. . 100
0x07	REG_FLAGS	Bitfeld mit Statusflags
0x08	REG_ELAPSED_MIN	Vergangene Minuten
0x09	REG_REMAINING_MIN	Restminuten
0x0A	REG_WATER_LEVEL	Bitfeld der Wasserstandssensoren

Schreibregister `0x10..0x13`

Adresse	Name	Bedeutung
---	---	---
0x10	REG_COMMAND	Kommando
0x11	REG_CMD_PARAM1	Parameter 1
0x12	REG_CMD_PARAM2	Parameter 2
0x13	REG_CMD_PARAM3	Parameter 3

5. Zustände und Fehlercodes

`REG\_STATUS`

Wert	Zustand
---	---
0	IDLE
1	DOOR_CHECK
2	FILL_HEAT
3	WASHING
4	DRAIN1
5	SPINNING
6	DRAIN2
7	DONE
8	ERROR
9	PAUSED

`REG\_ERROR`

Wert	Fehler
---	---
0	ERR_NONE
1	ERR_OVERHEAT
2	ERR_SENSOR_FAIL
3	ERR_FILL_TIMEOUT
4	ERR_HEAT_TIMEOUT
5	ERR_DRAIN_TIMEOUT
6	ERR_MOTOR_STALL
7	ERR_MOTOR_STOP_FAIL
8	ERR_WATER_AT_START

ERR_WATER_AT_START bedeutet: Beim Start lag bereits Wasser oberhalb des LOW-Sensors in der Trommel.

6. Statusbits

`REG\_FLAGS`

Bit	Maske	Bedeutung
---	---	---
0	0x01	Tuer verriegelt
1	0x02	Heizung aktiv
2	0x04	Pumpe aktiv

Bit	Maske	Bedeutung
3	0x08	Motor aktiv
4	0x10	Wasserzulauf offen
5	0x20	ESP32-Verbindung aktiv
6	0x40	Fehler aktiv
7	0x80	Programm laeuft

REG_WATER_LEVEL

Bit	Maske	Bedeutung
---	---	---
0	0x01	LOW-Sensor aktiv: Wasser steht mindestens auf LOW-Level
1	0x02	HIGH-Sensor aktiv: Wasser steht mindestens auf HIGH-Level

Praxis:

- 0x00: Trommel leer, LOW-Level unterschritten
- 0x01: Wasser ueber LOW-Level, aber unter HIGH-Level
- 0x03: HIGH-Level erreicht

Fuer das Basisprogramm bedeutet das:

- Fuellen endet, sobald REG_WATER_LEVEL & 0x02 gesetzt ist.
- Abpumpen laeuft, bis REG_WATER_LEVEL & 0x01 geloescht ist.
- Danach pumpt der Controller noch 5 s weiter.
- Erst nach diesem Nachlauf und bei stehendem Motor wird die Tuer entriegelt.

7. Kommandos

Wert	Name	Wirkung
---	---	---
0x00	CMD_NOP	keine Aktion
0x01	CMD_START	Programm starten
0x02	CMD_STOP	Programm abbrechen und nach IDLE
0x03	CMD_PAUSE	Laufendes Programm pausieren
0x04	CMD_RESUME	Pause verlassen
0x05	CMD_SET_TEMP	Zieltemperatur setzen, PARAM1 = degC
0x06	CMD_SET_RPM	Ziel-RPM setzen, PARAM2:PARAM3 = uint16
0x07	CMD_RESET_ERROR	Fehler quittieren und nach IDLE

Hinweis:

- Ein "Programm" vom ESP32 besteht aktuell aus optionalen Overrides wie CMD_SET_TEMP und CMD_SET_RPM sowie anschliessend CMD_START.
- Es gibt derzeit kein separates Rezept- oder Ablauf-Register fuer vollstaendige Waschprogramme.

8. ESPHome-Beispiel

Status lesen:

yaml

```
interval:
  - interval: 500ms
    then:
      - lambda: |-
          uint8_t data[11];
          auto err = id(i2c_bus).write(0x20, (const uint8_t[]){0x00}, 1);
          if (err != i2c::ERROR_OK) return;
          err = id(i2c_bus).read(0x20, data, 11);
          if (err != i2c::ERROR_OK) return;

          const uint8_t flags = data[7];
          const uint8_t water = data[10];
```

Temperatur setzen:

yaml

```
- lambda: |-
    uint8_t cmd[] = {0x10, 0x05, 40, 0x00, 0x00};
    id(i2c_bus).write(0x20, cmd, 5);
```

RPM setzen:

yaml

```
- lambda: |-
    uint16_t rpm = 1400;
    uint8_t cmd[] = {0x10, 0x06, 0x00, (uint8_t)(rpm >> 8), (uint8_t)(rpm & 0xFF)};
    id(i2c_bus).write(0x20, cmd, 5);
```

Start:

yaml

```
- lambda: |-
    uint8_t cmd[] = {0x10, 0x01};
    id(i2c_bus).write(0x20, cmd, 2);
```